

Trương Nhật Đăng – 23200037

Link source code và các ảnh được dùng:

https://github.com/kamengai1512-prog/BTVN_Buoi2/tree/main/BTVN2

 manhattan_distance.cpp	26-Sep-26 3:25 PM	C++ Source File	3 KB
 manhattan_distance.exe	26-Sep-26 3:25 PM	Application	387 KB
 pped.cpp	26-Sep-26 3:18 PM	C++ Source File	8 KB
 aped_feature_vectors.txt	26-Sep-26 3:15 PM	Text Document	6 KB
Earlier this week			
 pped.exe	24-Sep-26 10:35 PM	Application	390 KB
 rgb2gray.exe	24-Sep-26 9:26 PM	Application	373 KB
 config.txt	24-Sep-26 9:26 PM	Text Document	1 KB
 rgb2gray.cpp	24-Sep-26 9:14 PM	C++ Source File	3 KB
 jpg2raw.py	24-Sep-26 5:17 PM	Python File	2 KB
 pped_images	24-Sep-26 9:58 PM	File folder	
 grey_images	24-Sep-26 9:26 PM	File folder	
 raw_images	24-Sep-26 9:26 PM	File folder	
 images	24-Sep-26 5:15 PM	File folder	

Ta có cấu trúc thư mục như trên ứng với các yêu cầu đề bài. 1 file images để chứa 10 ảnh khuôn mặt và 3 file còn lại chứa ảnh được chuyển từ file images gốc hoặc có nguồn gốc từ nó.

1. Chuyển đổi định dạng & Trích xuất kích thước (jpg2raw.py)

- Ý tưởng:

- Ảnh màu gốc thường ở định dạng nén JPG. Để phục vụ việc xử lý mức điểm ảnh (pixel-level) hiệu quả trong C/C++, cần chuyển đổi dữ liệu pixel màu về dạng thô (RAW - chuỗi byte liên tục).
- Vì các ảnh đầu vào có độ phân giải không đồng nhất, kích thước (Chiều rộng W , Chiều cao H) của từng ảnh được trích xuất và lưu vào file config.txt để làm thông số đầu vào cho các chương trình C xử lý động.

```
import os
import numpy as np
from PIL import Image

# 1. Tên thư mục đầu vào và đầu ra
input_folder = "images"      # Thư mục chứa 10 ảnh JPG ban đầu
output_folder = "raw_images" # Thư mục chứa 10 file .raw đầu ra

# 2. Mở file config.txt để ghi kích thước [width height] của 10 ảnh
with open("config.txt", "w") as f:
    f.write("10 10\n")
```

```

for k in range(1, 11):
    # Đường dẫn ảnh đầu vào: images/face1.jpg, ...
    input_jpg = os.path.join(input_folder, f"face{k}.jpg")

    # Đường dẫn file đầu ra: raw_images/face1.raw, ...
    output_raw = os.path.join(output_folder, f"face{k}.raw")

    # Đọc ảnh JPG
    img_jpg = Image.open(input_jpg).convert("RGB")
    img_array = np.array(img_jpg, dtype=np.uint8)

    # Lấy kích thước ảnh
    height, width, _ = img_array.shape

    # Ghi kích thước riêng của từng ảnh vào config.txt
    f_cfg.write(f"{width} {height}\n")

    # Xuất mảng dữ liệu ra file .raw trong thư mục raw_images
    img_array.tofile(output_raw)
    print(f'Da xuất: {input_jpg} -> {output_raw} ({width}x{height})")

print("\nHoan thanh!/.")

```

File config.txt có nội dung là: WxH

```

486 705
447 447
501 640
399 501
1080 1440
600 806
447 447
387 516
480 360
780 438

```

2. Chuyển đổi ảnh màu RGB sang ảnh xám Grayscale (rgb2gray.cpp)

- Ý tưởng:

- Ảnh RGB có 3 kênh màu (Red, Green, Blue). Để phát hiện biên và trích xuất đặc trưng, ta sử dụng độ sáng (Luminance) thay vì màu sắc nhằm giảm độ phức tạp tính toán.
- Sử dụng công thức chuyển đổi chuẩn Luminance (ITU-R BT.601):
$$\text{Gray} = 0.299 \times R + 0.587 \times G + 0.114 \times B$$
- Mỗi pixel được rút gọn từ 3 byte màu thành 1 byte độ xám (8-bit, giá trị từ 0 → 255).

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int widths[10];

```

```

int heights[10];

// 1. Đọc kích thước của 10 ảnh từ file config.txt
FILE *f_cfg = fopen("config.txt", "r");

for (int k = 0; k < 10; k++) {
    if (fscanf(f_cfg, "%d %d", &widths[k], &heights[k]) != 2) {
        printf("Lỗi: Thieu du lieu kích thước o dòng %d trong config.txt\n", k + 1);
        fclose(f_cfg);
        return -1;
    }
}
fclose(f_cfg);
printf("Đã đọc xong config.txt.\n\n");

char input_filename[64];
char output_filename[64];

// 2. Vòng lặp chuyển đổi 10 ảnh RGB sang Grayscale
for (int k = 1; k <= 10; k++) {
    int width = widths[k - 1];
    int height = heights[k - 1];
    size_t num_pixels = (size_t)width * height;

    // Đường dẫn file
    sprintf(input_filename, "raw_images/face%d.raw", k);
    sprintf(output_filename, "grey_images/grey%d.raw", k);

    // Mở file RAW đầu vào
    FILE *f_in = fopen(input_filename, "rb");
    if (!f_in) {
        printf("Không tìm thấy file: %s\n", input_filename);
        continue;
    }

    // Cấp phát bộ nhớ theo đúng kích thước từng ảnh
    unsigned char *rgb_buf = (unsigned char *)malloc(num_pixels * 3);
    unsigned char *grey_buf = (unsigned char *)malloc(num_pixels);

    // Đọc dữ liệu RGB từ file .raw
    fread(rgb_buf, 1, num_pixels * 3, f_in);
    fclose(f_in);

    // Chuyển đổi RGB sang Grayscale (8-bit) bằng công thức Luminance
    for (size_t i = 0; i < num_pixels; i++) {
        unsigned char r = rgb_buf[i * 3 + 0];
        unsigned char g = rgb_buf[i * 3 + 1];
        unsigned char b = rgb_buf[i * 3 + 2];

        grey_buf[i] = (unsigned char)(0.299f * r + 0.587f * g + 0.114f * b);
    }

    // Ghi dữ liệu xám ra file grey.raw
    FILE *f_out = fopen(output_filename, "wb");
    if (f_out) {

```

```
fwrite(grey_buf, 1, num_pixels, f_out);
fclose(f_out);
printf("Thanh cong: %s -> %s (%dx%d)\n", input_filename, output_filename, width,
height);
} else {
    printf("Loi khi ghi file: %s\n", output_filename);
}

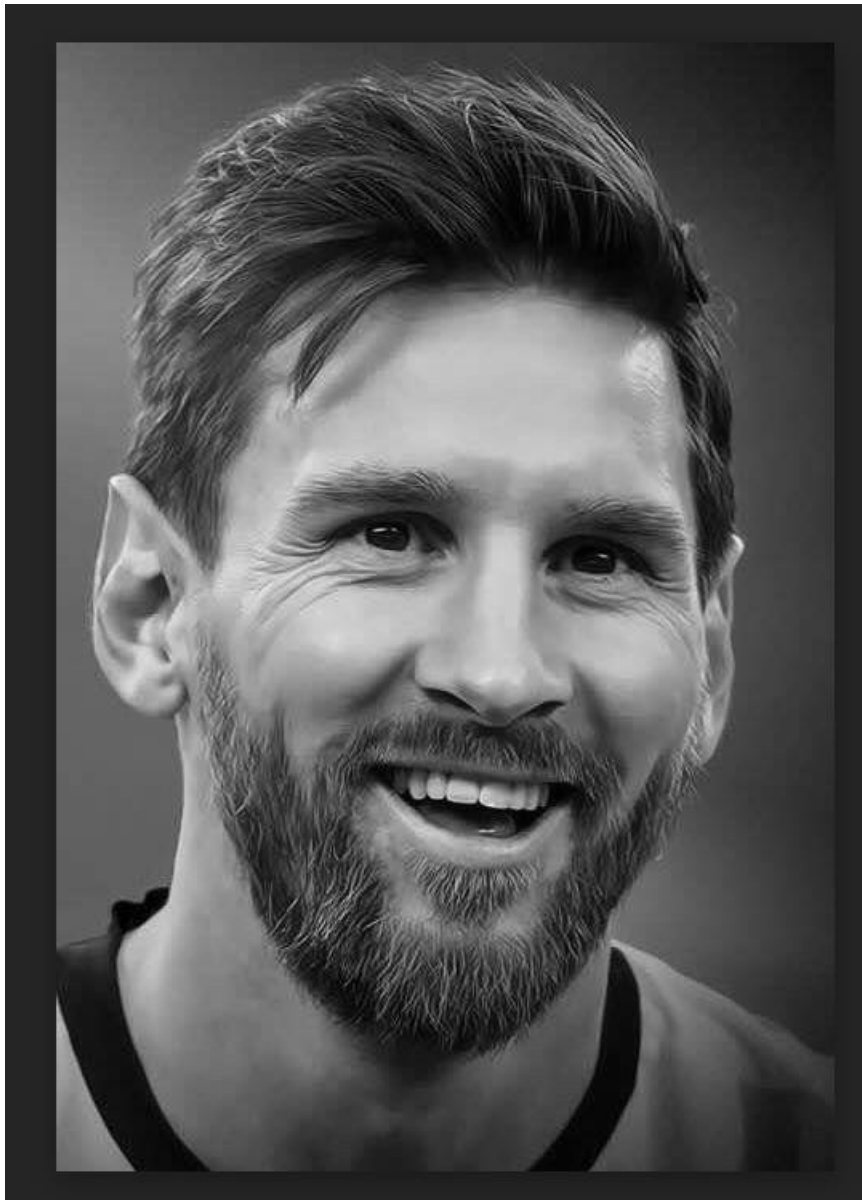
// Giải phóng bộ nhớ của ảnh hiện tại
free(rgb_buf);
free(grey_buf);
}

printf("\nHoan thanh chuyen doi 10 anh sang grayscale!\n");
return 0;
}
```

Kết quả: Từ



Thành:



3. Lọc biên PPED & Trích xuất Vector đặc trưng APED 64D (pped.cpp)

Thuật toán kết hợp giữa **PPED (Pixel-Pair Edge Detection)** và **APED (Adapted Pixel-pair Edge Direction)** để vừa tạo ảnh biên vừa trích xuất đặc trưng hình học:

a. Tính Ngưỡng thích nghi (Adaptive Thresholding via Median):

- Tại mỗi pixel $I(x, y)$ (trừ vùng biên ảnh 2 pixel):
 - Tính chênh lệch giá trị độ xám giữa các cặp pixel lân cận theo hướng **Ngang (H)** (20 cặp) và hướng **Độc (V)** (20 cặp) trong cửa sổ lân cận.
 - Lấy **Trung vị (Median)** của 40 giá trị chênh lệch tuyệt đối này để đại diện cho mức độ biến động cục bộ quanh pixel $I(x, y)$.
 - Ngưỡng thích nghi cục bộ được tính bằng:

$$TH(x, y) = 5.0 \times \text{Med}(x, y)$$

b. Xác định Điểm biên và Hướng biên chính:

- Tính độ chênh lệch theo 4 hướng xung quanh pixel (x, y) :
 - **Ngang** ($H - 0^\circ$): $I_H = |I(x, y + 1) - I(x, y - 1)|$
 - **Chéo phụ** ($P - 45^\circ$): $I_P = |I(x - 1, y + 1) - I(x + 1, y - 1)|$
 - **Đọc** ($V - 90^\circ$): $I_V = |I(y + 1, x) - I(y - 1, x)|$
 - **Chéo chính** ($M - -45^\circ$): $I_M = |I(y + 1, x + 1) - I(y - 1, x - 1)|$
- Tìm hướng có độ chênh lệch lớn nhất: $I_{\max} = \max(I_H, I_P, I_V, I_M)$.
- **Phân ngưỡng**: Nếu $I_{\max} > TH(x, y)$ và $I_{\max} > 20.0$ (loại bỏ nhiễu nền):
 - Pixel (x, y) được ghi nhận là điểm biên (gán giá trị 255 trên ảnh biên đầu ra).
 - Ghi nhận hướng biên tương ứng (H, P, V hoặc M).

c. Xây dựng Vector đặc trưng APED 64 chiều (64-Dimensional Feature Vector):

- Chia không gian ảnh thành lưới 4×4 gồm **16 ô (sub-blocks)** bằng nhau.
- Mỗi ô sẽ đếm số lượng điểm biên thuộc 4 hướng (H, P, V, M).
- Tổng cộng ta có: $16 \text{ ô} \times 4 \text{ hướng} = 64$ giá trị đếm.
- **Chuẩn hóa mật độ**: Giá trị của từng chiều trong vector được chia cho tổng số pixel của ảnh ($W \times H$) để đảm bảo đặc trưng không bị ảnh hưởng bởi kích thước ảnh:

$$\text{Vector}[i] = \frac{\text{Số điểm biên trong ô (theo hướng)}}{\text{Tổng số pixel của ảnh}}$$

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define NUM_IMAGES 10
#define VECTOR_DIM 64 // Vector 64 chiều (16 ô x 4 hướng)

// Hàm hỗ trợ sắp xếp mảng để tìm giá trị Trung vị (Median)
int compare(const void *a, const void *b) {
    float diff = *(float*)a - *(float*)b;
    return (diff > 0) - (diff < 0);
}

// Thuật toán PPED lọc biên đồng thời trích xuất Vector đặc trưng APED 64D
void pped_and_extract_aped_vector(const unsigned char *I, unsigned char *dst, int w, int h, float
vector[VECTOR_DIM]) {
    // 1. Khởi tạo ảnh đầu ra màu đen và xóa trắng vector 64 chiều
    for (int i = 0; i < w * h; i++) dst[i] = 0;
    for (int i = 0; i < VECTOR_DIM; i++) vector[i] = 0.0f;

    // Các mảng đếm điểm biên cho 4 hướng {H, P, V, M} tương ứng 16 ô
    float H_vec[16] = {0.0f}, P_vec[16] = {0.0f}, V_vec[16] = {0.0f}, M_vec[16] = {0.0f};

    // Tính kích thước ô lưới 4x4 linh hoạt theo W và H của ảnh gốc
    float cell_w = (float)w / 4.0f;
    float cell_h = (float)h / 4.0f;
```

```

// 2. Lặp qua từng pixel của ảnh gốc (tránh viền 2 pixel)
for (int y = 2; y < h - 2; y++) {
    for (int x = 2; x < w - 2; x++) {

        float diff_vals[40];
        int count = 0;

        // --- Công thức (4): Chênh lệch hướng Ngang H_nm ---
        for (int n = -2; n <= 1; n++) {
            for (int m = -2; m <= 2; m++) {
                float val1 = I[(y + m) * w + (x + n + 1)];
                float val2 = I[(y + m) * w + (x + n)];
                diff_vals[count++] = fabsf(val1 - val2);
            }
        }

        // --- Công thức (5): Chênh lệch hướng Dọc V_nm ---
        for (int n = -2; n <= 1; n++) {
            for (int m = -2; m <= 2; m++) {
                float val1 = I[(y + n + 1) * w + (x + m)];
                float val2 = I[(y + n) * w + (x + m)];
                diff_vals[count++] = fabsf(val1 - val2);
            }
        }

        // --- Tìm Trung vị Med(x,y) từ 40 giá trị ---
        qsort(diff_vals, 40, sizeof(float), compare);
        float Med = (diff_vals[19] + diff_vals[20]) / 2.0f;

        // --- Công thức (6): Tính Ngưỡng TH(x, y) ---
        float TH = Med * 5.0f;

        // --- TÍNH ĐỘ CHÊNH LỆCH THEO 4 HƯỚNG {H, P(45°), V, M(-45°)} ---
        float I_H = fabsf((float)I[y * w + (x + 1)] - (float)I[y * w + (x - 1)]);
        float I_P = fabsf((float)I[(y - 1) * w + (x + 1)] - (float)I[(y + 1) * w + (x - 1)]);
        float I_V = fabsf((float)I[(y + 1) * w + x] - (float)I[(y - 1) * w + x]);
        float I_M = fabsf((float)I[(y + 1) * w + (x + 1)] - (float)I[(y - 1) * w + (x - 1)]);

        // Tìm giá trị chênh lệch lớn nhất giữa 4 hướng và hướng tương ứng
        float I_max = I_H;
        int max_dir = 0; // 0: H, 1: P, 2: V, 3: M

        if (I_P > I_max) { I_max = I_P; max_dir = 1; }
        if (I_V > I_max) { I_max = I_V; max_dir = 2; }
        if (I_M > I_max) { I_max = I_M; max_dir = 3; }

        // --- Công thức (7): Phân ngưỡng xác định điểm biên ---
        if (I_max > TH && I_max > 20.0f) {
            dst[y * w + x] = 255; // Đánh dấu điểm biên lên ảnh đầu ra

            // --- TÍCH LŨY BIÊN VÀO CÁC Ô LƯỚI (Lưới 4x4 -> 16 Ô) ---
            int a = (int)(x / cell_w); // Chỉ số cột ô (0, 1, 2, 3)
            int b = (int)(y / cell_h); // Chỉ số hàng ô (0, 1, 2, 3)
        }
    }
}

```

```

// Giới hạn an toàn tránh viền ảnh
if (a > 3) a = 3;
if (b > 3) b = 3;

int idx = a + 4 * b; // Vị trí ô từ 0 đến 15

// Tăng số lượng đếm điểm biên cho hướng tương ứng tại ô `idx`
if (max_dir == 0) H_vec[idx] += 1.0f;
else if (max_dir == 1) P_vec[idx] += 1.0f;
else if (max_dir == 2) V_vec[idx] += 1.0f;
else if (max_dir == 3) M_vec[idx] += 1.0f;
} else {
    dst[y * w + x] = 0; // Không phải biên
}
}
}

// --- GHÉP 4 VECTOR THÀNH VECTOR 64 CHIỀU & CHUẨN HÓA MẬT ĐỘ ---
float total_pixels = (float)(w * h);
for (int i = 0; i < 16; i++) {
    vector[0 + i] = H_vec[i] / total_pixels; // 16 chiều Ngang H (0-15)
    vector[16 + i] = P_vec[i] / total_pixels; // 16 chiều Chéo P (16-31)
    vector[32 + i] = V_vec[i] / total_pixels; // 16 chiều Dọc V (32-47)
    vector[48 + i] = M_vec[i] / total_pixels; // 16 chiều Chéo M (48-63)
}
}

int main() {
    int widths[NUM_IMAGES], heights[NUM_IMAGES];

    // Đọc kích thước của 10 ảnh từ config.txt
    FILE *f_cfg = fopen("config.txt", "r");
    if (!f_cfg) {
        printf("Lỗi: Không mở được file config.txt!\n");
        return -1;
    }

    for (int k = 0; k < NUM_IMAGES; k++) {
        fscanf(f_cfg, "%d %d", &widths[k], &heights[k]);
    }
    fclose(f_cfg);

    // Mảng 2D chứa 10 Vector 64 chiều cho 10 bức ảnh
    float feature_vectors[NUM_IMAGES][VECTOR_DIM];
    char input_filename[64], output_filename[64];

    for (int k = 1; k <= NUM_IMAGES; k++) {
        int width = widths[k - 1];
        int height = heights[k - 1];
        size_t num_pixels = (size_t)width * height;

        // Đường dẫn file ảnh xám đầu vào và file ảnh biên đầu ra
        sprintf(input_filename, "grey_images/grey%d.raw", k);
        sprintf(output_filename, "pped_images/pped%d.raw", k);
    }
}

```

```

FILE *f_in = fopen(input_filename, "rb");
if (!f_in) {
    printf("Khong mo duoc file: %s\n", input_filename);
    continue;
}

unsigned char *gray_buf = (unsigned char *)malloc(num_pixels);
unsigned char *pped_buf = (unsigned char *)malloc(num_pixels);

fread(gray_buf, 1, num_pixels, f_in);
fclose(f_in);

// Lọc biên PPED + Trích xuất Vector APED 64D đồng thời
pped_and_extract_aped_vector(gray_buf, pped_buf, width, height, feature_vectors[k - 1]);

// Ghi file ảnh biên PPED ra đĩa
FILE *f_out = fopen(output_filename, "wb");
if (f_out) {
    fwrite(pped_buf, 1, num_pixels, f_out);
    fclose(f_out);
    printf("Xử lý thành công: %s -> %s (%dx%d)\n", input_filename, output_filename, width,
height);
}

free(gray_buf);
free(pped_buf);
}

// --- GHI KẾT QUẢ 10 VECTOR ĐẶC TRƯNG 64 CHIỀU RA FILE RIÊNG ---
FILE *f_vec = fopen("aped_feature_vectors.txt", "w");
if (f_vec) {
    for (int k = 0; k < NUM_IMAGES; k++) {
        fprintf(f_vec, "Anh_%d: ", k + 1);
        for (int j = 0; j < VECTOR_DIM; j++) {
            fprintf(f_vec, "%.6f ", feature_vectors[k][j]);
        }
        fprintf(f_vec, "\n");
    }
    fclose(f_vec);
    printf("\nHoàn thành! Đã trích xuất & lưu 10 Vector 64D vào file
'aped_feature_vectors.txt'\n");
} else {
    printf("Lỗi: Không thể tạo file aped_feature_vectors.txt để ghi vector!\n");
}

return 0;
}

```



Features Vector:

Anh_1: 0.000178 0.000204 0.000464 0.000225 0.000916 0.000207 0.000128 0.001004 0.000835 0.001150 0.000893 0.000625 0.000070 0.000581 0.001057 0.000035 0.000260 0.001725 0.002204 0.000797 0.001486 0.002373 0.001579 0.003254 0.002221 0.002703 0.003628 0.001045 0.002376 0.000986 0.001506 0.000677 0.000301 0.001416 0.000388 0.000055 0.000198 0.001127 0.000406 0.000248 0.000058 0.000283 0.000604 0.000073 0.000067 0.000123 0.000125 0.000018 0.000969 0.002244 0.000887 0.000207 0.003324 0.003628 0.001077 0.001316 0.001783 0.002390 0.002779 0.001448 0.000884 0.000768 0.003339 0.000079
Anh_2: 0.001692 0.001081 0.000651 0.000606 0.000761 0.000515 0.001246 0.001757 0.000195 0.000285 0.001156 0.001266 0.000265 0.000876 0.001051 0.000611 0.002888 0.004454 0.003063 0.002683 0.002592 0.003613 0.003298 0.005510 0.001992 0.001391 0.001627 0.002597 0.001516 0.002688 0.001426 0.000480 0.000070 0.000265 0.000230 0.000090 0.000160 0.000616 0.000290 0.000275 0.000265 0.000305 0.000170 0.000160 0.000681 0.000641 0.000265 0.000100 0.004024 0.000706 0.000330 0.000180 0.002242 0.001697 0.001692 0.002983 0.001967 0.000736 0.001541 0.003608 0.002277 0.001937 0.001912 0.003453
Anh_3: 0.000206 0.000234 0.000555 0.000172 0.000511 0.000359 0.000434 0.000496 0.000458 0.000274 0.000474 0.000458 0.000561 0.000209 0.000324 0.000327 0.000362 0.001403 0.001347 0.000443 0.000627 0.000745 0.000976 0.000879 0.001207 0.000879 0.001229 0.000792 0.000861 0.000490 0.000851 0.001500 0.000237 0.001054 0.000586 0.000125 0.000172 0.000521 0.000402

```

0.000225 0.000178 0.000624 0.000536 0.000087 0.000153 0.000312 0.000421 0.000458 0.000692
0.001488 0.000876 0.000250 0.000621 0.000914 0.000957 0.000487 0.000508 0.000830 0.001310
0.001017 0.001276 0.000221 0.000873 0.001737
Anh_4: 0.000265 0.000150 0.000140 0.000520 0.000000 0.000460 0.000415 0.001106 0.000000
0.000045 0.000270 0.000240 0.000020 0.000325 0.000730 0.000795 0.000400 0.000775 0.003447
0.001576 0.000000 0.001801 0.005873 0.003882 0.000000 0.000265 0.003767 0.000930 0.000110
0.000875 0.004142 0.001986 0.000000 0.000350 0.001196 0.000115 0.000000 0.000135 0.001746
0.000320 0.000000 0.000000 0.000495 0.000115 0.000000 0.000010 0.000485 0.000395 0.000320
0.001596 0.002831 0.000880 0.000000 0.001816 0.006008 0.004107 0.000000 0.000325 0.004712
0.002441 0.000205 0.001761 0.005693 0.002221
Anh_5: 0.000027 0.000068 0.000093 0.000003 0.000014 0.000177 0.000276 0.000010 0.000004
0.000098 0.000115 0.000004 0.000141 0.000093 0.000109 0.000070 0.000092 0.001085 0.001665
0.000009 0.000091 0.001544 0.001731 0.000008 0.000156 0.001315 0.000776 0.000465 0.001062
0.000833 0.000867 0.000631 0.000000 0.000138 0.000057 0.000000 0.000001 0.000167 0.000110
0.000000 0.000048 0.000079 0.000113 0.000010 0.000104 0.000140 0.000157 0.000065 0.000109
0.001400 0.000374 0.000005 0.000057 0.001971 0.001836 0.000014 0.000674 0.000854 0.001566
0.000073 0.000734 0.000819 0.000804 0.000583
Anh_6: 0.000428 0.000378 0.000300 0.000130 0.000817 0.000486 0.000693 0.000312 0.000821
0.000684 0.000854 0.000232 0.000602 0.000416 0.000645 0.000529 0.000517 0.000809 0.000908
0.000143 0.001981 0.001313 0.001466 0.000515 0.002971 0.001344 0.001210 0.000449 0.001780
0.001445 0.001187 0.000490 0.000174 0.000490 0.000523 0.000054 0.000263 0.000629 0.000527
0.000031 0.000296 0.000544 0.000507 0.000225 0.000215 0.000565 0.000199 0.000252 0.000980
0.000906 0.000804 0.000134 0.002215 0.001046 0.001373 0.000645 0.001501 0.001305 0.001675
0.000525 0.001022 0.000788 0.002196 0.000794
Anh_7: 0.000060 0.000310 0.000390 0.000005 0.000175 0.000450 0.000596 0.000130 0.000290
0.000265 0.000065 0.000070 0.000235 0.000290 0.000225 0.000325 0.000671 0.001061 0.001912
0.000150 0.000901 0.002152 0.001546 0.000691 0.001476 0.001697 0.000355 0.001181 0.003483
0.000636 0.002172 0.001752 0.000095 0.000475 0.000531 0.000035 0.000240 0.000255 0.000310
0.000095 0.000215 0.000225 0.000220 0.000055 0.000510 0.000240 0.000671 0.000295 0.000776
0.001687 0.001066 0.000160 0.001306 0.001151 0.002072 0.000375 0.001862 0.001091 0.001872
0.000320 0.002087 0.001832 0.002983 0.002653
Anh_8: 0.001067 0.000441 0.000376 0.000426 0.000951 0.000401 0.000366 0.000326 0.000611
0.000270 0.000300 0.000696 0.000366 0.000476 0.000521 0.000676 0.002314 0.002268 0.001938
0.002274 0.001828 0.001287 0.001963 0.002774 0.001833 0.002524 0.001958 0.002694 0.001172
0.001933 0.001763 0.002048 0.000401 0.000776 0.000846 0.000305 0.000551 0.000476 0.000436
0.000551 0.000376 0.000135 0.000220 0.000636 0.000320 0.000220 0.000280 0.000606 0.004297
0.002073 0.002163 0.001107 0.003195 0.001958 0.001207 0.001307 0.003505 0.001307 0.002509
0.002574 0.001362 0.001602 0.002424 0.001512
Anh_9: 0.000012 0.000156 0.000087 0.000093 0.000017 0.000405 0.000220 0.000110 0.000046
0.000260 0.000336 0.000104 0.000029 0.001227 0.000602 0.000255 0.000498 0.003160 0.002731
0.000330 0.000903 0.004323 0.002234 0.001192 0.001852 0.005521 0.004421 0.001892 0.000422
0.004086 0.004068 0.003189 0.000000 0.001024 0.000324 0.000000 0.000127 0.000347 0.000191
0.000110 0.000203 0.000741 0.000272 0.000226 0.000046 0.000255 0.000249 0.000069 0.000046
0.003623 0.001169 0.000660 0.000995 0.004352 0.002622 0.000978 0.001910 0.005272 0.002888
0.001678 0.001898 0.005747 0.002535 0.000700
Anh_10: 0.001262 0.001958 0.001911 0.001982 0.001262 0.001449 0.001563 0.002456 0.001068
0.001478 0.001598 0.002210 0.000729 0.000837 0.000893 0.000249 0.001996 0.006876 0.005775
0.002178 0.002350 0.003469 0.002705 0.003182 0.001531 0.002798 0.002242 0.002517 0.000978
0.002298 0.000562 0.000969 0.000471 0.001112 0.000840 0.000237 0.000492 0.001144 0.000834
0.000202 0.000304 0.000738 0.000553 0.000155 0.000316 0.000609 0.000503 0.000173 0.001783
0.006132 0.004932 0.001832 0.001505 0.003697 0.003223 0.002400 0.000925 0.002113 0.002365
0.002435 0.001402 0.001232 0.002520 0.000281

```

4. So khớp ảnh bằng Khoảng cách Manhattan (manhattan_distance.cpp)

- **Ý tưởng:**

- Để so sánh độ tương đồng giữa ảnh truy vấn (Query Image) và các ảnh trong cơ sở dữ liệu, ta so sánh sự tương đồng giữa các vector đặc trưng 64 chiều của chúng.
- Sử dụng **Khoảng cách Manhattan (L_1 Distance)**, đo tổng độ chênh lệch tuyệt đối giữa từng tọa độ của 2 vector $\vec{u}$ và $\vec{v}$:

$$D(\vec{u}, \vec{v}) = \sum_{i=0}^{63} |u_i - v_i|$$

- **Nhận dạng:**

- Khoảng cách D càng nhỏ, hai ảnh càng tương đồng về mặt hình học và phân bố biên.
- Ảnh trong cơ sở dữ liệu có khoảng cách Manhattan nhỏ nhất (min D) tới ảnh truy vấn sẽ được kết luận là ảnh cần tìm.

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define NUM_IMAGES 10
#define VECTOR_DIM 64

// Hàm tính khoảng cách Manhattan giữa 2 vector 64 chiều
float manhattan_distance(const float v1[VECTOR_DIM], const float v2[VECTOR_DIM]) {
    float dist = 0.0f;
    for (int i = 0; i < VECTOR_DIM; i++) {
        dist += fabsf(v1[i] - v2[i]); // Tổng trị tuyệt đối hiệu từng chiều
    }
    return dist;
}

int main() {
    float vectors[NUM_IMAGES][VECTOR_DIM];

    // 1. Đọc dữ liệu 10 vector 64D từ file aped_feature_vectors.txt
    FILE *f_in = fopen("aped_feature_vectors.txt", "r");

    for (int k = 0; k < NUM_IMAGES; k++) {
        char dummy[32];
        // Đọc và bỏ qua nhãn "Anh_X:" ở đầu mỗi dòng
        if (fscanf(f_in, "%s", dummy) != 1) break;

        // Đọc 64 giá trị float của từng vector
        for (int j = 0; j < VECTOR_DIM; j++) {
            fscanf(f_in, "%f", &vectors[k][j]);
        }
    }
    fclose(f_in);

    // 2. Nhập vị trí ảnh bạn muốn dùng làm Query (từ 1 đến 10)
    int query_id = 5; //Vd: 5
```

```

int query_idx = query_id - 1; // Chuyển về chỉ số mảng (0-9)
float *query_vector = vectors[query_idx];

printf("\n=== TINH 10 KHOANG CACH MANHATTAN CHO ANH %d ===\n", query_id);

float distances[NUM_IMAGES];
float min_dist = 1e9f; // Khởi tạo min ban đầu rất lớn
int min_idx = -1;

// 3. Tính khoảng cách Manhattan tới cả 10 ảnh trong CSDL và tìm Min
for (int k = 0; k < NUM_IMAGES; k++) {
    distances[k] = manhattan_distance(query_vector, vectors[k]);

    printf("Khoang cach toi Anh_%-2d : %.6f", k + 1, distances[k]);
    if (k == query_idx) {
        printf(" <-- ");
    }
    printf("\n");

    // Cập nhật giá trị Min
    if (distances[k] < min_dist) {
        min_dist = distances[k];
        min_idx = k;
    }
}

// 4. In kết quả Min tìm được
printf("-----\n");
printf("KHOANG CACH MIN TRONG 10 ANH: %.6f\n", min_dist);
printf("--> ANH CAN TIM: Anh_%.d\n", min_idx + 1);

return 0;
}

```

```
=== TINH 10 KHOANG CACH MANHATTAN CHO ANH 5 ===
```

```
Khoang cach toi Anh_1 : 0.047016  
Khoang cach toi Anh_2 : 0.067906  
Khoang cach toi Anh_3 : 0.025793  
Khoang cach toi Anh_4 : 0.059072  
Khoang cach toi Anh_5 : 0.000000 <--  
Khoang cach toi Anh_6 : 0.030762  
Khoang cach toi Anh_7 : 0.030177  
Khoang cach toi Anh_8 : 0.057182  
Khoang cach toi Anh_9 : 0.061244  
Khoang cach toi Anh_10 : 0.087479
```

```
-----  
KHOANG CACH MIN TRONG 10 ANH: 0.000000
```

```
--> ANH CAN TIM: Anh_5
```

```
-----  
Process exited after 0.04037 seconds with return value 0  
Press any key to continue . . . |
```